

۱. مسترکی: یکی یا دوتا؟

- **مسترکی را خودت تعریف می کنی، از جایی “نمی گیری”.**

موقع اجرای Meilisearch با `--master-key` هر رشته ای که بدهی می شود مسترکی آن سرور.

- **لوکال و پروداکشن دو سرور جدا هستند ⇒ بهتر است دو مسترکی جدا داشته باشی.**

مثلا:

- لوکال: `MEILI\_MASTER\_KEY\_LOCAL\_123`

- سرور: `MEILI\_MASTER\_KEY\_PROD\_987`

- از نظر فنی می توانی یک کلید مشترک هم بگذاری، ولی **بهتر و امن تر این است که برای هر محیط یک کلید جدا داشته باشی** و در `env` همان محیط تنظیمش کنی:

لوکال (`zanburak-backend/.env`):

env``

SCOUT_DRIVER=meilisearch

MEILISEARCH_HOST=http://127.0.0.1:7700

MEILISEARCH_KEY=MEILI_MASTER_KEY_LOCAL_123

...

سرور (پروداکشن، `env` روی اوبونتو):

env``

SCOUT_DRIVER=meilisearch

MEILISEARCH_HOST=http://127.0.0.1:7700 # اگر Meili روی همان سرور است

MEILISEARCH_KEY=MEILI_MASTER_KEY_PROD_987

...

۲. نصب و اجرا روی ویندوز (لوکال)

فرض می‌کنیم Docker نداری و می‌خواهی مستقیماً اجرا کنی.

۱. **دانلود باینری ویندوز**

- از سایت Meilisearch (بخش Releases) نسخه Windows (`.zip` یا `.exe`) را دانلود کن.

- فایل را در یک پوشه مثل `C:\meilisearch` قرار بده.

۲. **اجرای Meilisearch در PowerShell**

در PowerShell:

```
powershell``
```

```
cd C:\meilisearch
```

```
"meilisearch.exe --master-key "MEILI_MASTER_KEY_LOCAL_123\"
```

```
``
```

- بعد از اجرا باید ببینی که نوشته روی `http://127.0.0.1:7700` در حال گوش دادن است.

- همین کافیه؛ لاراولت با تنظیمات `env` که بالا نوشتم وصل می‌شود.

- برای لوکال معمولاً همین که دستی اجراش کنی کافی است؛ لازم نیست حتماً سرویس دائمی بسازی (مگر این که خودت بخواهی).

۳. نصب و اجرا روی سرور اوبونتو ۲۴.۰۴

۳.۱. نصب باینری

روی سرور (SSH):

```
bash``
```

برو به یک پوشه موقت

```
cd /tmp
```

دانلود آخرین نسخه لینوکس x86_64 (طبق داکيومنت Meilisearch)

```
curl -L https://install.meilisearch.com | sh
```

باینری ساخته شده را به /usr/local/bin/ منتقل کن

```
sudo mv meilisearch /usr/local/bin/meilisearch
```

دسترسی اجرا

```
sudo chmod +x /usr/local/bin/meilisearch
```

```
``
```

(اگر از اسکریپت راحت نیستی، می‌شود مستقیماً از صفحه Releases فایل `meilisearch-linux-amd64` را دانلود و همین کار را انجام داد.)

۳.۲. ساخت کاربر جدا برای Meilisearch (اختیاری ولی خوب)

```
bash``
```

```
sudo useradd -r -s /bin/false meili
```

```
``
```

۳.۳. ساخت سرویس systemd برای استارت خودکار

فایل سرویس:

```
bash``
```

```
sudo nano /etc/systemd/system/meilisearch.service
```

```
``
```

محتوا را بگذار (در صورت نیاز مسیرها و کلید را عوض کن):

```
ini``
```

```
[Unit]
```

```
Description=Meilisearch
```

```
After=network.target
```

```
[Service]
```

```
User=meili
```

```
Group=meili
```

```
\ ExecStart=/usr/local/bin/meilisearch
```

```
\ "http-addr "127.0.0.1:7700--
```

```
"master-key "MEILI_MASTER_KEY_PROD_987--
```

```
Restart=always
```

```
RestartSec=5
```

```
"Environment="MEILI_ENV=production
```

```
[Install]
```

```
WantedBy=multi-user.target
```

```
``
```

سپس:

```
bash``
```

```
# ری‌لود systemd
```

```
sudo systemctl daemon-reload
```

```
# فعال‌سازی برای استارت خودکار بعد از ری‌بوت
```

```
sudo systemctl enable mailsearch
```

```
# استارت سرویس
```

```
sudo systemctl start mailsearch
```

```
# چک وضعیت
```

```
sudo systemctl status mailsearch
```

```
``
```

- الان هر بار سرور ری‌بوت شود، **Meilsearch** خودش اتومات بالا می‌آید**.

- چون `--http-addr "127.0.0.1:7700"` است، فقط خود سرور (لاراول) می‌تواند وصل شود (امن‌تر است). اگر خواستی از بیرون هم دسترسی داشته باشی، باید آن را به `۰.۰.۰.۰:۷۷۰۰` تغییر بدهی و با فایروال/ریورس‌پروکسی مدیریت کنی.

۴. جمع‌بندی سریع

- **کلید لوکال و پروداکشن الزماً یکی نیست؛ بهتر است دو تا جدا تعریف کنی** و در هر محیط در `env.` خود آن محیط تنظیم‌شان کنی.

- ویندوز: فقط باینری را دانلود کن و با `meilsearch.exe --master-key\..."` اجرا کن.

- اوبونتو ۲۴.۰۴: باینری را در `/usr/local/bin/` بگذار، یک `meilsearch.service` بساز، `systemctl enable/start` کن تا **بعد از ری‌بوت هم خودکار بالا بیاید**.

- بعدش در هر محیط یک بار `php artisan scout:import "App\Models\Course"` و همین برای `Episode` و `Question` را بزن تا ایندکس‌ها ساخته شوند.

اگر خواستی، در قدم بعدی می‌توانم برات یک کانفیگ کمی امن‌تر هم پیشنهاد بدهم (مثلاً استفاده از API key های read-only برای فرانت، لاگ‌ها، و محدود کردن دسترسی شبکه).